

x86-64 vs ARMv8

A Case Study in ISA Design

made by nithilan rameshkumar, bl.sc.u4cse24031

Introduction

x86-64 is the 64-bit version of x86 and is common in PCs and servers. It keeps compatibility with older software, so the design is a bit complex.

ARMv8 is ARM’s 64-bit ISA. It is widely used in phones and is now also popular in laptops and cloud systems.

1 · Architecture Type

Property	x86-64	ARMv8
ISA Family	CISC	RISC
Encoding	Variable-length	Fixed 32-bit
Main Focus	Compatibility + performance	Efficiency + simplicity
Common Use	PCs and servers	Mobile and newer laptops

x86-64 uses a CISC style with more complex instructions. ARMv8 follows RISC and keeps instructions simpler and more regular.

2 · Registers

x86-64 Registers

x86-64 has **16 general-purpose 64-bit registers** (RAX to R15). It also has smaller views (EAX, AX, AL), which come from older x86 design.

Register Set	Count	Notes
GP (64-bit)	16	RAX-R15
SIMD/FP	16	XMM0-XMM15
Flags	1	RFLAGS
Instruction Ptr	1	RIP

ARMv8 Registers

ARMv8 has **31 general-purpose 64-bit registers** (X0-X30) plus SP and XZR. The register model is cleaner than x86-64.

Register Set	Count	Notes
GP (64-bit)	31	X0-X30
Zero Reg	1	XZR
Stack Pointer	1	SP
SIMD/FP	32	V0-V31
Flags	1	NZCV

Main difference: ARM has more GP registers (31 vs 16), so it usually needs fewer memory spills.

3 · Instruction Sizes

x86-64	ARMv8																
Instructions are variable-length (1 to 15 bytes). This gives flexibility but makes decoding harder.	Instructions are fixed 32 bits (4 bytes) in ARMv8, so decoding is easier and instruction boundaries are clear.																
<table><tr><th>Length</th><th>Example</th></tr><tr><td>1 B</td><td>NOP</td></tr><tr><td>2-3 B</td><td>Simple instructions</td></tr><tr><td>4-8 B</td><td>Memory instructions</td></tr><tr><td>Up to 15 B</td><td>Complex/vector instructions</td></tr></table>	Length	Example	1 B	NOP	2-3 B	Simple instructions	4-8 B	Memory instructions	Up to 15 B	Complex/vector instructions	<table><tr><th>Length</th><th>Notes</th></tr><tr><td>32 bits</td><td>All ARMv8 instructions</td></tr><tr><td>16/32 bits</td><td>Older ARM modes used mixed sizes</td></tr></table>	Length	Notes	32 bits	All ARMv8 instructions	16/32 bits	Older ARM modes used mixed sizes
Length	Example																
1 B	NOP																
2-3 B	Simple instructions																
4-8 B	Memory instructions																
Up to 15 B	Complex/vector instructions																
Length	Notes																
32 bits	All ARMv8 instructions																
16/32 bits	Older ARM modes used mixed sizes																

So, x86-64 is flexible but complex; ARMv8 is simpler for hardware decode.

4 · Pipelining and Hazards

Overview

Both use modern pipelining and out-of-order execution in real CPUs.

Aspect	x86-64	ARMv8
Decode	Complex, variable-length	Simpler, fixed-length
Register handling	Legacy aliases exist	Cleaner model
Power use	Usually higher	Usually better
Performance	Very strong in high-end CPUs	Strong and efficient

Data Hazards

Both use techniques like forwarding and register renaming to reduce stalls. x86-64 has extra complexity from partial registers.

Control Hazards

Both depend on branch prediction. Wrong predictions cause pipeline flushes and performance loss.

Summary

Dimension	x86-64	ARMv8
ISA class	CISC	RISC
GP Registers	16	31
Instr. size	1-15 bytes	Fixed 32 bits
Decode	Harder	Easier
Power efficiency	Medium	High

x86-64 is still excellent for compatibility and high-end desktop/server use. ARMv8 is simpler and often more power-efficient, which is why it is growing fast in phones, laptops, and cloud.